SE1020 · OBJECT ORIENTED PROGRAMMING PROJECT

CRUD by Team Member

Movie Ticket Booking System · Java Spring Boot, SQL database and HTML user interface

TEAM MEMBERS AND STUDENT IDS

MEMBER 1

IT26100914

User & Authentication

11 CRUD operations

MEMBER 2

IT26100915

Movie Catalogue

5 CRUD operations

MEMBER 3

IT26100916

Theatre & Screen

8 CRUD operations

MEMBER 4

IT26100918

Showtime Scheduling

5 CRUD operations

MEMBER 5

IT26100919

Booking & Seat Reservation

6 CRUD operations

MEMBER 6

IT26100920

Payment & File Handling

9 CRUD operations

Each member owns one component and performs Create, Read, Update and Delete operations. Customer and admin operations are listed separately.

CRUD by Team Member – SE1020 Movie Ticket Booking

Each member owns one component. Every component has its own entity, DTO, repository, service, controller and UI, and performs **Create, Read, Update and Delete** (CRUD). Customer and admin operations are listed separately because they use separate login portals and separate endpoints.

For each operation below:

- **Endpoint** is the API route.
- **What it does** explains the operation in plain words.
- **Rules** lists the checks the service enforces.
- **Why** explains the design reason.

Full API details are in README section 5.

MEMBER 1

User & Authentication

IT26100914

Classes: `User` (abstract), `Customer`, `Admin`, `UserRole`, `CustomerService`, `AdminService`, `AuthService`, the auth strategies (`AuthStrategy`, `AbstractAuthStrategy`, `CustomerAuthStrategy`, `AdminAuthStrategy`), `CustomerProfileController`, `AdminUserController`, `AuthController`.

CUSTOMER

CREATE **Create – register** `POST /api/auth/customer/register`

WHAT IT DOES	creates a new customer account with name, email, password and optional phone.
RULES	the email must be unique (case-insensitive). The password is stored only as a BCrypt hash.
WHY	customers register themselves, so no admin is needed to start booking.

READ **Read – profile** `GET /api/customer/profile`

WHAT IT DOES	returns the logged-in customer's own details, including loyalty points.
RULES	the customer is identified from the session, so a customer cannot read another customer's profile.
WHY	the session decides whose data is returned, so the URL never contains an ID to tamper with.

READ **Read – customer list (admin)** `GET /api/admin/customers?q=`

WHAT IT DOES	lists all customers, or filters them by name or email when `q` is given.
RULES	admin role only.
WHY	admins need to look up a customer for support or for the delete check.

READ **Read – one customer (admin)** `GET /api/admin/customers/{id}`

WHAT IT DOES	returns one customer by ID.
RULES	admin role only. Returns 404 if the customer does not exist.

UPDATE **Update – profile** `PUT /api/customer/profile`

WHAT IT DOES	changes the full name and phone. It can also change the password if a new one is given.
RULES	the email cannot be changed after registration. The password is changed only when a new one is supplied.
WHY	the email is the login name, so keeping it fixed prevents accidental lock-outs.

DELETE **Delete – own account** `DELETE /api/customer/profile`

WHAT IT DOES	deletes the logged-in customer's account and ends the session.
RULES	blocked if the customer has any booking history.
WHY	deleting an account with bookings would break the booking and payment records. This is the same rule as the admin delete below.

DELETE **Delete – customer (admin)** `DELETE /api/admin/customers/{id}`

WHAT IT DOES	deletes a customer account.
RULES	blocked if the customer has booking history. Returns an error message explaining why.

ADMIN

CREATE **Create – add admin** `POST /api/admin/admins`

WHAT IT DOES	creates a new admin account with a full name, email, password (at least 6 characters) and permission level, SUPER_ADMIN or STAFF.
RULES	only a SUPER_ADMIN can add admins. The email must be unique. The permission level must be one of the two allowed values.
WHY	only senior staff should be able to grant admin access.

READ **Read – admin list** `GET /api/admin/admins`

WHAT IT DOES	lists all admin accounts with their permission levels.
RULES	admin role only.

UPDATE **Update – edit admin** `PUT /api/admin/admins/{id}`

WHAT IT DOES	changes an admin's permission level (SUPER_ADMIN or STAFF).
RULES	only a SUPER_ADMIN can edit admins. Name, email and password are not changed through this endpoint.
WHY	the permission level is the only setting an operator needs to change, so the update stays small and safe.

DELETE **Delete – remove admin** `DELETE /api/admin/admins/{id}`

WHAT IT DOES	removes an admin account.
RULES	only a SUPER_ADMIN can delete admins. An admin cannot delete their own account, so the system always keeps a way to log in.

Login (separate portals) `POST /api/auth/customer/login` , `POST /api/auth/admin/login`

- **What it does:** checks the email and password against the matching account type.
- **Why:** customers and admins use different pages and different endpoints, so a customer account cannot open the admin dashboard. `AuthInterceptor` checks the role on every request to `/api/customer/**` and `/api/admin/**` .

UI pages: `customer-register.html` , `customer-login.html` , `profile.html` , `admin-login.html` , and the Customers and Admins sections of `admin-dashboard.html` .

MEMBER 2

Movie Catalogue

IT26100915

Classes: `Movie` (abstract), `StandardMovie` , `ImaxMovie` , `ThreeDMovie` , `MovieFactory` , `MovieRepository` , `MovieService` , `MovieController` , `MovieDto` .

CREATE **Create** `POST /api/admin/movies`

WHAT IT DOES	adds a movie with its title, genre, language, duration, rating, release date, description, format and details (director, producers, writers, cast, trailer link, scene images).
RULES	the title must be unique (case-insensitive). The duration must be at least 30 minutes. The format must be STANDARD, IMAX or THREE_D.
WHY	`MovieFactory` creates the correct subclass for the format, so the ticket surcharge (IMAX +LKR 300, 3D +LKR 200, Standard 0) is decided by the object type, not by an `if` statement.

READ **Read – list** `GET /api/movies` `title` `genre`

WHAT IT DOES	returns the movie list, optionally filtered by title or genre. Public, so visitors can browse without logging in.
WHY	the catalogue is the first thing a customer sees.

READ **Read – details** `GET /api/movies/{id}`

WHAT IT DOES	returns one movie with its credits, trailer link and scene images, used by `movie.html` .
WHY	the details page needs everything about one film in a single request.

UPDATE **Update** `PUT /api/admin/movies/{id}`

WHAT IT DOES	changes the movie's details.
RULES	the format cannot be changed after creation. The error message tells the admin to create a new movie instead.
WHY	JPA cannot change an object's subclass in place. Changing the format would also change the surcharge that existing bookings were priced with.

DELETE **Delete** `DELETE /api/admin/movies/{id}`

WHAT IT DOES	removes a movie.
RULES	blocked while any showtime still uses the movie.
WHY	deleting a movie with showtimes would leave those showtimes without a film.

UI pages: `index.html` (catalogue), `movie.html` (details, credits, trailer and scenes, showtimes), and the Movies section of the admin dashboard.

File link: `FileService.importMovies()` reads `data/seed/movies.txt` and creates movies through the same service, so the same rules apply to imported films.

MEMBER 3

Theatre & Screen

IT26100916

Classes: `Theatre`, `Screen` (abstract), `StandardScreen`, `PremiumScreen`, `VipScreen`, `ScreenFactory`, `TheatreService`, `ScreenService`, `TheatreController`, `ScreenController`, `TheatreDto`, `ScreenDto`.

THEATRE

CREATE **Create** `POST /api/admin/theatres`

WHAT IT DOES	adds a theatre with its name, city, address and phone.
WHY	each screen belongs to a theatre, so theatres are created first.

READ **Read** `GET /api/theatres` `GET /api/theatres/{id}`

WHAT IT DOES	lists theatres or returns one theatre. Public.
WHY	customers choose a theatre before they choose a showtime.

UPDATE **Update** `PUT /api/admin/theatres/{id}`

WHAT IT DOES	changes the theatre's name, city or address.
RULES	no restriction, because the theatre's details do not affect bookings directly.

DELETE **Delete** `DELETE /api/admin/theatres/{id}`

WHAT IT DOES	removes a theatre.
RULES	blocked while the theatre still has screens.
WHY	removing a theatre with screens would leave those screens without a building.

SCREEN

CREATE **Create** `POST /api/admin/screens`

WHAT IT DOES	adds a screen to a theatre with its name, screen type, number of rows and seats per row.
RULES	the type must be STANDARD, PREMIUM or VIP. Rows are letters (maximum 26). Seats per row are numbers (maximum 30).
WHY	`ScreenFactory` creates the correct subclass for the type. Each type has its own price multiplier (Standard ×1.00, Premium ×1.30, VIP ×1.80), chosen by polymorphism.

READ **Read** `GET /api/screens?theatreId=` `GET /api/screens/{id}`

WHAT IT DOES	lists the screens of a theatre, or returns one screen.
WHY	the showtime form needs the screens of a chosen theatre.

UPDATE **Update** `PUT /api/admin/screens/{id}`

WHAT IT DOES	changes the screen's name, theatre, number of rows or seats per row.
RULES	the type cannot be changed after creation. The seat layout cannot change while the screen has showtimes.
WHY	the type fixes the price multiplier that existing showtimes use. Changing the seats while bookings exist could make booked seats disappear.

DELETE **Delete** `DELETE /api/admin/screens/{id}`

WHAT IT DOES	removes a screen.
RULES	blocked while the screen has showtimes.

UI pages: `theatres.html` (public list of theatres and screens), and the Theatres and Screens sections of the admin dashboard.

MEMBER 4

Showtime Scheduling

IT26100918

Classes: `Showtime`, `ShowtimeRepository`, `ShowService`, `ShowController`, `ShowDto`, `PricingStrategy` (interface), `WeekdayPricingStrategy`, `WeekendPricingStrategy`, `MatineePricingStrategy`.

CREATE **Create** `POST /api/admin/shows`

WHAT IT DOES	schedules a movie on a screen at a start time with a base price. The end time is calculated from the movie's duration.
RULES	• The start time must be in the future. • No other show on the same screen may overlap this time. The check uses the start and end times of both shows.
WHY	a screen can only show one film at a time, and the overlap check prevents double-booking the room.

READ **Read – public list** `GET /api/shows?movieId=&date=` `GET /api/shows/{id}`

WHAT IT DOES	returns the showtimes for a movie or a date, with the seat price and the list of seats already held.
WHY	customers need to see upcoming times and how many seats are left.

READ **Read – admin list** `GET /api/admin/shows`

WHAT IT DOES	returns all showtimes, including past ones, for the admin dashboard.

UPDATE **Update** `PUT /api/admin/shows/{id}`

WHAT IT DOES	changes the movie, screen, start time or base price.
RULES	the same future-time and overlap checks apply, excluding this showtime itself, so it does not clash with its own old time.
WHY	an admin may need to move a show, and the check must still apply after the change.

DELETE **Delete** `DELETE /api/admin/shows/{id}`

WHAT IT DOES	removes a showtime.
RULES	blocked once any booking exists for the showtime.
WHY	the bookings and payments depend on this showtime's price and time.

Pricing rule (not a CRUD operation, but part of this component):

- Seat price = base price × screen multiplier + movie surcharge.
- Then the rule with the highest priority applies: matinee before 12:00 (×0.80), weekend Saturday and Sunday (×1.20), otherwise weekday (×1.00).
- **Why:** each rule is a separate class that implements the same `PricingStrategy` interface, so a new rule can be added without changing the code that calls them.

UI pages: the showtime list in `movie.html`, and the Showtimes section of the admin dashboard.

MEMBER 5

Booking & Seat Reservation

IT26100919

Classes: `Booking`, `BookingStatus`, `BookingRepository`, `BookingService`, `BookingController`, `BookingDto`, `CancellationPolicy` (interface), `FullRefundPolicy`, `PartialRefundPolicy`, `NoRefundPolicy`.

CREATE Create `POST /api/customer/bookings`

WHAT IT DOES reserves chosen seats for a showtime. The booking starts as PENDING_PAYMENT and gets a reference code (BK followed by 8 characters).

RULES
- The showtime must not have started yet.
- At least one seat is required, and at most 10 seats per booking.
- Seat codes must be valid (a letter row and a number, inside the screen's layout).
- A seat cannot be chosen twice in the same request.
- A seat already held by a PENDING_PAYMENT or CONFIRMED booking cannot be taken.

WHY the reservation holds the seats until payment, so two customers cannot buy the same seat.

READ Read – my bookings `GET /api/customer/bookings` `GET /api/customer/bookings/{id}`

WHAT IT DOES lists the logged-in customer's bookings (newest first), or returns one booking.

RULES a customer can only see their own bookings. Another customer's booking returns "not found".

WHY the booking ID alone must not reveal someone else's booking.

READ Read – all bookings (admin) `GET /api/admin/bookings`

WHAT IT DOES lists every booking, newest first.

UPDATE Update – change seats `PUT /api/customer/bookings/{id}`

WHAT IT DOES replaces the seats of a booking and recalculates the total.

RULES allowed only while the booking is PENDING_PAYMENT. The same seat checks as creation apply, ignoring this booking's own old seats.

WHY after payment the seats are fixed, because the payment amount was already taken.

DELETE Delete – cancel (customer) `DELETE /api/customer/bookings/{id}`

WHAT IT DOES cancels the booking.
- **Awaiting payment:** the booking is deleted and the seats are released. No money was taken.
- **Confirmed:** the cancellation policy decides the refund, the payment is refunded by that amount, and the booking is marked CANCELLED.

RULES a booking that is already cancelled cannot be cancelled again.

REFUND POLICY FOR CONFIRMED BOOKINGS

Refund policy for confirmed bookings:
- 24 hours or more before the show: 100% refund (`FullRefundPolicy`).
- 2 to 24 hours before the show: 50% refund (`PartialRefundPolicy`).
- Under 2 hours before the show: no refund (`NoRefundPolicy`).

WHY the policy is a separate class for each rule, so the refund rules can change without touching the cancel logic.

DELETE **Delete – remove cancelled booking (admin)** `DELETE /api/admin/bookings/{id}`

WHAT IT DOES	removes a cancelled booking and its payment records.
RULES	only CANCELLED bookings can be deleted. Active bookings are protected.
WHY	this keeps the sales records clean while still protecting bookings that customers are using.

UI pages: `booking.html` (seat map, edit mode for changing seats), `my-bookings.html` (list and cancel), and the Bookings section of the admin dashboard.

MEMBER 6

Payment & File Handling

IT26100920

Classes: `Payment`, `PaymentStatus`, `PaymentMethodType`, `PaymentRepository`, `PaymentService`, `PaymentController`, `PaymentDto`, `PaymentProcessor` (interface), `AbstractPaymentProcessor`, `CardPaymentProcessor`, `WalletPaymentProcessor`, `CashCounterPaymentProcessor`, `FileService`, `FileController`, `FileDto`.

CREATE **Create – pay** `POST /api/customer/payments`

WHAT IT DOES	pays for a PENDING_PAYMENT booking. The booking becomes CONFIRMED, and the customer earns one loyalty point per seat.
RULES	• The booking must belong to the logged-in customer and must be awaiting payment. • The payment method decides the validation and fee: • **Card:** 16 digits, 2% fee. Only the last four digits are stored. • **Wallet:** ID of 5 to 40 characters, no fee. • **Cash counter:** code in the form `CNT-NN`, no fee.
WHY	each method is a separate processor class that extends `AbstractPaymentProcessor`. The abstract class has a fixed sequence in `charge()` (template method): validate the reference, calculate the fee, create the transaction reference, then build the receipt. Each subclass supplies only the parts that differ, such as its validation rule and fee. Callers use only `charge()`, through the `PaymentProcessor` interface.

READ **Read – my payments** `GET /api/customer/payments`

WHAT IT DOES	lists the customer's own payment history, newest first.

READ **Read – all payments (admin)** `GET /api/admin/payments`

WHAT IT DOES	lists all payments, newest first. This is the transaction history, so payments on cancelled bookings stay in the list (marked CANCELLED).
RULES	the sales analytics (dashboard totals, daily chart, payment methods, top movies, CSV export and the sales report) leave out payments on cancelled bookings. Only the history list shows them.

UPDATE **Update – manual refund (admin)** `PUT /api/admin/payments/{id}`

WHAT IT DOES	records a full manual refund, for example cash returned at the counter. The payment becomes REFUNDED and the booking is CANCELLED.
RULES	only the REFUNDED status is accepted here. A payment that is already fully refunded is rejected.
WHY	automatic refunds happen during customer cancellation. This endpoint covers refunds that the system did not make itself.

DELETE **Delete – remove refunded payment (admin)** `DELETE /api/admin/payments/{id}`

WHAT IT DOES	removes a payment record.
RULES	only REFUNDED payments can be deleted. Successful payments are kept as records.
WHY	a paid transaction must stay in the ledger, so only refunded ones can be removed.

FILE HANDLING

READ **Read – import movies** `POST /api/admin/files/import-movies` `data/seed/movies.txt`

WHAT IT DOES	reads . Each line is one movie with 14 pipe-separated fields. Movies whose title already exists are skipped.

READ **Read – import users** `POST /api/admin/files/import-users` `data/seed/users.txt`

WHAT IT DOES	reads . Each line is one account. Passwords are hashed when imported.

WRITE **Write – sales report** `GET /api/admin/files/export-sales` `data/reports/`

WHAT IT DOES	writes a timestamped report to and returns it as a download. Money totals exclude payments on cancelled bookings.

WRITE **Write – seed files kept in step** `data/seed/movies.txt` `data/seed/showtimes.txt`

WHAT IT DOES	after an admin adds, edits or deletes a movie or showtime, or is rewritten from the database. A file error is logged and does not undo the saved change.

- **Data storage:** the application data is stored in an H2 file database (`data/moviebooking.mv.db`), which needs no installation. MySQL or MariaDB can be used instead with the `mysql` profile. The seed text files are used for the import and export described above.
- **Why:** the course requires file reading and writing. The import uses the same services as the web forms, so imported data follows the same rules.

UI pages: the payment step in `booking.html` , and the Payments and Files & Reports sections of the admin dashboard.

Summary

Member	Component	Create	Read	Update	Delete
Member 1 **IT26100914**	User & Authentication	Customer register; admin add (SUPER_ADMIN)	Own profile; admin lists customers and admins	Own profile; admin edit (SUPER_ADMIN)	Own account; admin deletes customer (no bookings); admin deletes admin (SUPER_ADMIN)
Member 2 **IT26100915**	Movie Catalogue	Admin adds movie	Movie list and details	Admin edits movie (format fixed)	Admin deletes movie (no showtimes)
Member 3 **IT26100916**	Theatre & Screen	Admin adds theatre and screen	Theatre and screen lists	Admin edits theatre and screen (type fixed)	Admin deletes theatre (no screens) and screen (no showtimes)
Member 4 **IT26100918**	Showtime Scheduling	Admin adds showtime (overlap check)	Public list and admin list	Admin edits showtime (same checks)	Admin deletes showtime (no bookings)
Member 5 **IT26100919**	Booking & Seats	Customer books seats	My bookings; admin sees all	Customer changes seats (pending payment only)	Customer cancels (refund policy); admin deletes cancelled bookings
Member 6 **IT26100920**	Payment & Files	Customer pays (card, wallet or cash counter)	My payments; admin sees all	Admin records a manual refund	Admin deletes refunded payments; file import and sales-report export